



# Distributed Systems

## Design Issues Issue 1: Time in Distributed Systems



### An Efficient Implementation of Vector Clocks

Singhal and A. Kshemkalyani

#### Observation

When the number of processes are high only few processes are likely to frequently interact

**=> Between two successive sending events from process  $P_i$  to process  $P_j$ , only a few of the vector clock's entries are expected to change.**

- Piggyback only the information relative to the entries that changed.
- Send pairs (PID, Vector clock of PID)

### An Efficient Implementation of Vector Clocks

- When  $P_j$  receives a message with compressed timestamp

$$[(i_1, v_1), (i_2, v_2), \dots, (i_{n1}, v_{n1})]$$

it updates its vector clock as:

$$\rightarrow vt_j[i_k] = \max(vt_j[i_k], v_k) \quad \text{for all } k \text{ in message}$$

- Each process to remember the vector timestamp in the message last sent to every other process
- Storage overhead (direct implementation) –  $O(n^2)$
- Requires FIFO channels

### An Efficient Implementation of Vector Clocks

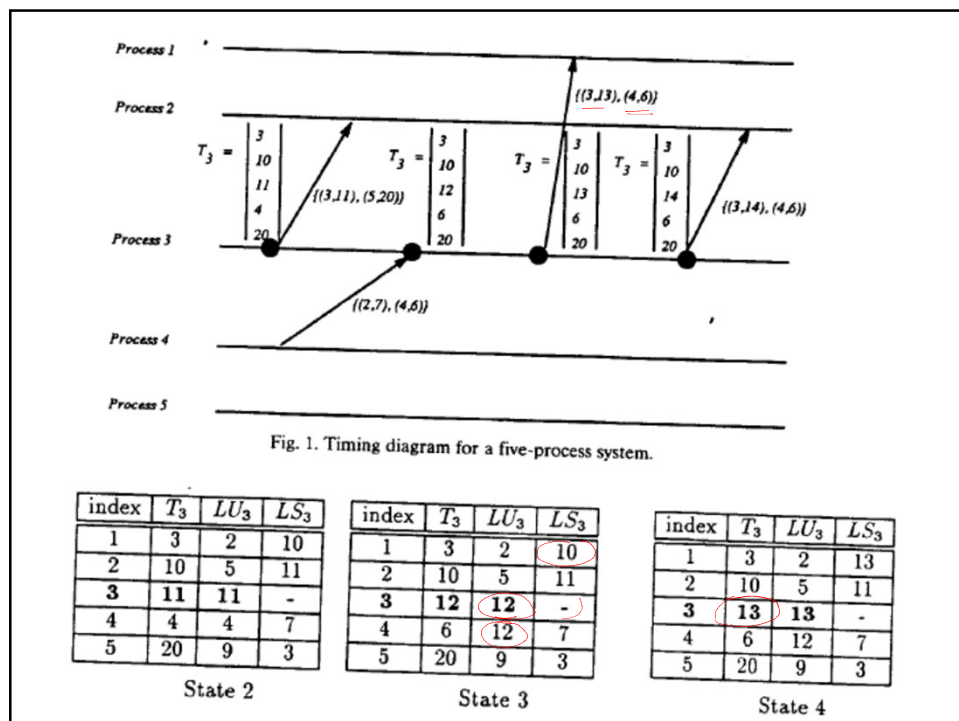
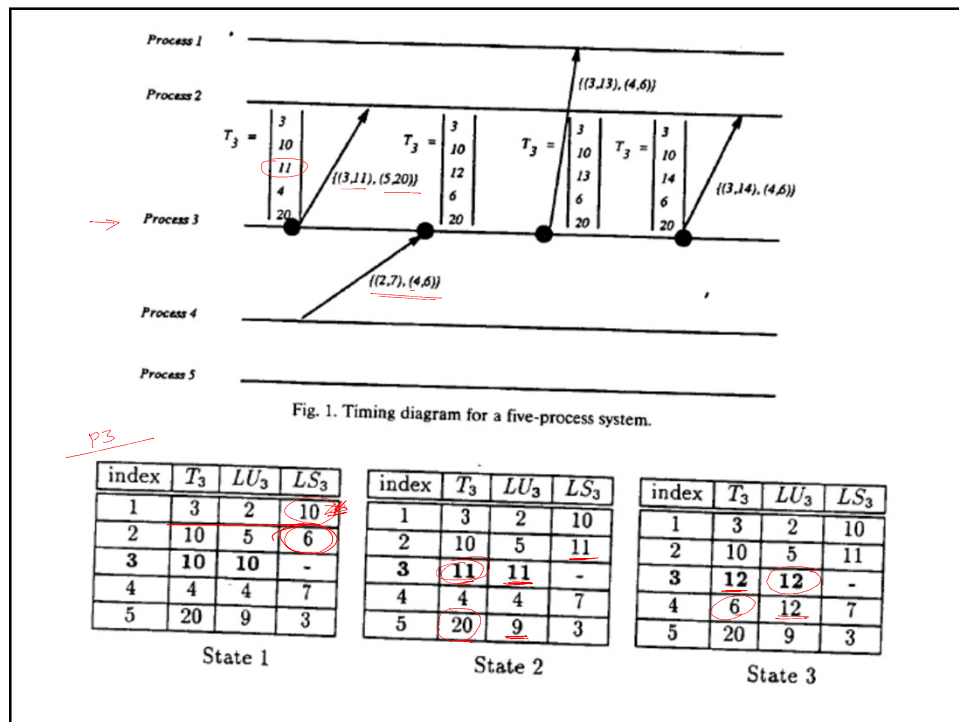
- A process  $p_i$  maintains two additional vectors:

$LS_i[1..n]$ :  $LS_i[j]$  indicates the value of  $vt_i[i]$  when process  $p_i$  last sent a message to processes  $p_j$

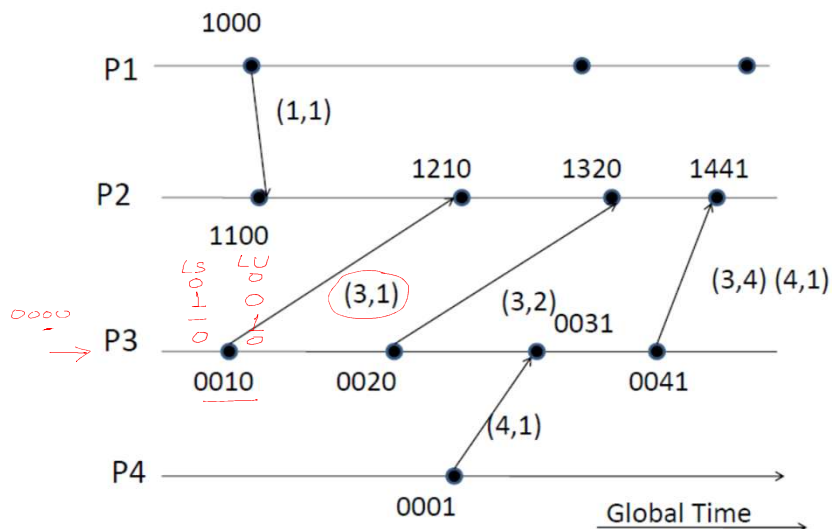
$LU_i[1..n]$ :  $LU_i[j]$  indicates the value of  $vt_i[i]$  when process  $p_i$  last updated the entry  $vt_i[j]$

- Storage overhead at each process?  $O(n)$
- When  $p_i$  sends a message to  $p_j$  it sends only a set of tuples

$$\{(x, vt_i[x]) \mid LS_i[j] < LU_i[x]\}$$



## An Efficient Implementation of Vector Clocks



KEE

7

INDIAN INSTITUTE OF TECHNOLOGY ROORKEE



## Distributed Systems Client-Server Computing TCP Server

— Unix  
— C

Ref: Unix Network Programming, The Socket Networking API by W Richard Stevens



## TCP

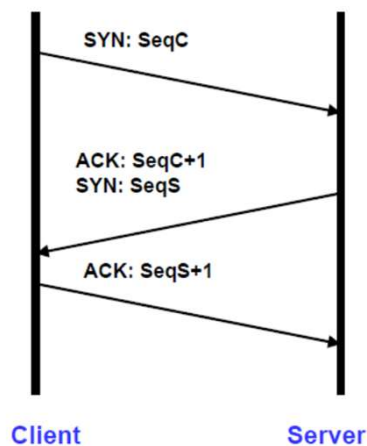


- Connection oriented
- **Reliable in-order** delivery of byte stream
- Full-duplex: each endpoint may send and receive
- Flow control
  - To ensure that sender does not **overrun receiver** by sending too fast
- Congestion control
  - Keep the sender from **overrunning the network**
  - Many simultaneous connections across routers (cross traffic)

IIT ROORKEE

9

## Establishing Connection: Three-Way handshake



- Each byte in byte stream is numbered (32-bit value)
- Each side notifies other of starting sequence number it will use for sending
- Why not simply chose 0?

IIT ROORKEE

10

## Initial Sequence Number Selection



### RFC 793

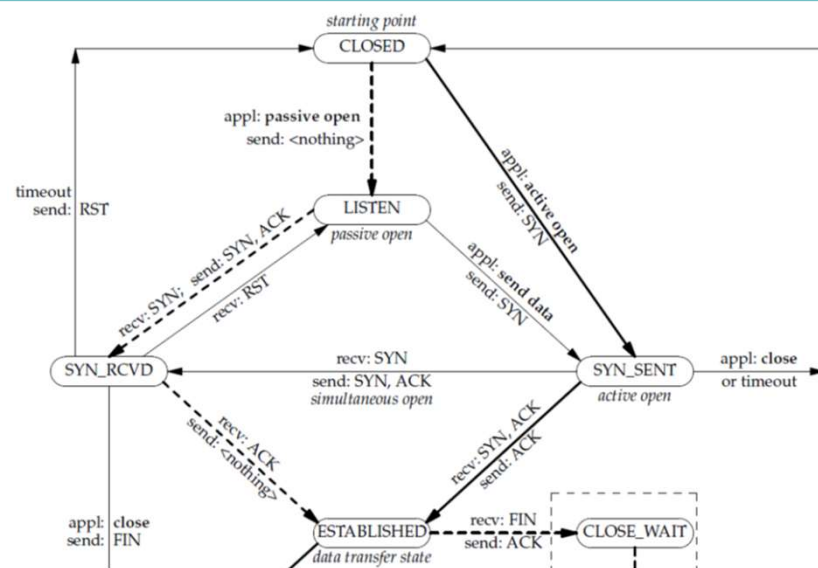


- When new connections are created, an initial sequence number (ISN) generator is employed which selects a new **32 bit ISN**.
- The generator is bound to a (possibly fictitious) 32 bit clock whose low order bit is **incremented roughly every 4 microseconds**.   
*Handwritten note:  $4 \times 55 \times 60 \times 60 \times 10^6$*
- Thus, the ISN **cycles approximately every 4.55 hours**.
- It is assumed that Maximum Segment Lifetime (MSL) is less than 4.55 hours

IIT ROORKEE

11

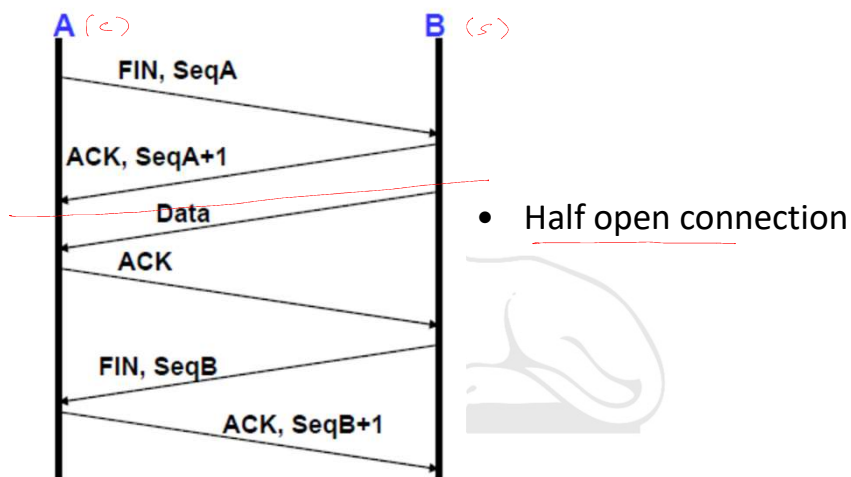
## TCP State Diagram: Connection Setup



IIT ROORKEE

12

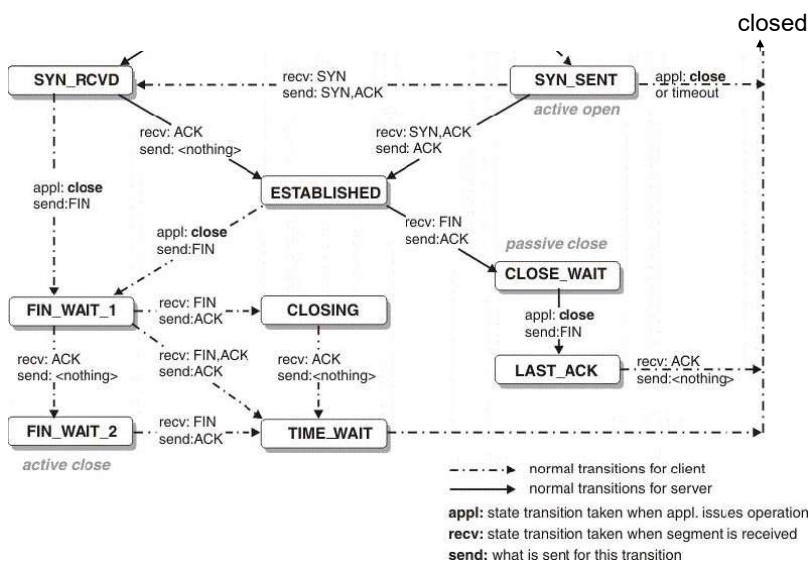
## Tearing Down Connection



IIT ROORKEE

13

## TCP: Connection Termination



IIT ROORKEE

14

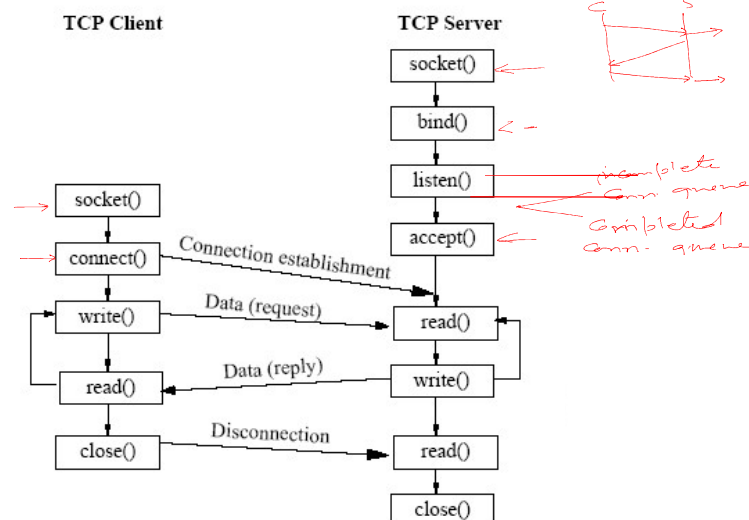
## TCP: Design Issues

- How big should the packets/segments be?
- What if packets are delivered out of order?
  - May take different paths through the network
- What if a packet is lost?
  - Packets may be dropped in the network
- What if a packet is corrupted in transit?
  - Detect error and fix it or resend
- How fast should the sender send?
  - Flow Control
  - Keeping the Pipe Full

IIT ROORKEE

15

## Socket system calls for connection-oriented protocol



IIT ROORKEE

16

## Socket API



- Most socket functions require a pointer to a socket address structure as an argument

- Each supported protocol suite defines its own socket data structure

|      |              |   |
|------|--------------|---|
| IPv4 | sockaddr_in  | ✓ |
| IPv6 | sockaddr_in6 | ✓ |
| XNS  | sockaddr_ns  | ✓ |
| Unix | sockaddr_un  | ✓ |

- Use a generic structure to represent all type of network addresses

```
struct sockaddr {
    uint8_t      sa_len;
    sa_family_t  sa_family;
    char         sa_data[14];
};
```

*16 bytes*

- size?

IIT ROORKEE

17

- For Internet addresses, we use sockaddr\_in:

```
struct sockaddr_in {
    uint8_t      sin_len;           /* length of structure */
    sa_family_t  sin_family;        /* set to AF_INET */
    in_port_t    sin_port;          /* Port number */
    struct in_addr sin_addr;         /* Contains the IP address */
    char         sin_zero[8];
};

struct in_addr {
    in_addr_t    s_addr; /* IP address in network ordering */
};
```

- IP Addressing and Name Resolution

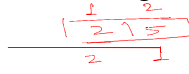
sin family: used to indicate which type of address is following.

Always set it to AF\_INET.

sin port: port number, in **network byte ordering**.

sin addr.s addr: IP address, in **network byte ordering**.

- To represent an unspecified IP address, set it to htonl(INADDR\_ANY).  
This tells the kernel to choose the IP address.

- The Internet protocols use big-endian byte ordering (network byte order) 
- Functions to convert between network byte order and host byte order
  - htons, htonl, ntohs, ntohl
- Address conversion functions
  - inet\_aton, inet\_ntoa, inet\_addr
- Problems with inet\_addr function
  - int inet\_aton(const char \*s, struct in\_addr \*addr);
  - in\_addr\_t inet\_addr(const char \*s);

-1  
255.255.255.255

o Want to do a network I/O?

o Get a file descriptor

int **socket**(int family, int type, int protocol);

family      AF\_UNIX / AF\_LOCAL ←  
               AF\_INET ←  
               AF\_NS  
               AF\_INET6 ←

Type        SOCK\_STREAM ←  
               SOCK\_DGRAM ←  
               SOCK\_RAW ← ICMP, IGMP, ...

Protocol    typically set to 0

- Not all combinations of family and type are valid
- On Linux allocating a socket means allocating a VFS inode